




# Online Marketplace: A Benchmark for Data Management in Microservices

RODRIGO LAIGNER, University of Copenhagen, Denmark
ZHEXIANG ZHANG, University of Copenhagen, Denmark
YIJIAN LIU, University of Copenhagen, Denmark
LEONARDO FREITAS GOMES, Amadeus, France
YONGLUAN ZHOU, University of Copenhagen, Denmark

Microservice architectures have become a popular approach for designing scalable distributed applications. Despite their extensive use in industrial settings for over a decade, there is limited understanding of the data management challenges that arise in these applications. Consequently, it has been difficult to advance data system technologies that effectively support microservice applications. To fill this gap, we present *Online Marketplace*, a microservice benchmark that highlights core data management challenges that existing benchmarks fail to address. These challenges include transaction processing, query processing, event processing, constraint enforcement, and data replication. We have defined criteria for various data management issues to enable proper comparison across data systems and platforms. Through case studies with state-of-the-art data platforms, we discuss the issues encountered while implementing and meeting *Online Marketplace*'s criteria. By capturing the overhead of meeting the key data management requirements that are overlooked by existing benchmarks, we gain actionable insights into the experimental platforms. This highlights the significance of *Online Marketplace* in advancing future data systems to meet the needs of microservice practitioners.

CCS Concepts: • **Information systems** → **Data management systems**; • **Computer systems organization** → **Distributed architectures**.

Additional Key Words and Phrases: microservices, data management, benchmark, online marketplace

**ACM Reference Format:**
Rodrigo Laigner, Zhexiang Zhang, Yijian Liu, Leonardo Freitas Gomes, and Yongluan Zhou. 2025. Online Marketplace: A Benchmark for Data Management in Microservices. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 3 (February 2025), 26 pages. https://doi.org/10.1145/3709653

## 1 Introduction

Microservice architecture has emerged in the last decade as a popular architectural style in industry settings. This style promotes the decomposition of an application into independent microservices with associated private states. From an organizational point of view, these principles allow different teams to manage and evolve their own modules independently. At the same time, it enables new modules to be introduced and deprecated modules to be removed without impacting the application as a whole. From a technological point of view, each module can be independently deployed on distributed computational resources, allowing for failure isolation and high availability. Meanwhile, the message-based communication paradigm serves as a powerful abstraction for triggering tasks

Authors' Contact Information: Rodrigo Laigner, University of Copenhagen, Copenhagen, Denmark, rnl@di.ku.dk; Zhexiang Zhang, University of Copenhagen, Copenhagen, Denmark, gjm699@alumni.ku.dk; Yijian Liu, University of Copenhagen, Copenhagen, Denmark, liu@di.ku.dk; Leonardo Freitas Gomes, Amadeus, Nice, France, leonardo.gomes@amadeus.com; Yongluan Zhou, University of Copenhagen, Copenhagen, Denmark, zhou@di.ku.dk.


This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/2-ART3
https://doi.org/10.1145/3709653

in remote microservices, facilitating data replication among microservices, and enabling failure recovery by replaying past events [39].

This industrial popularity has prompted cloud providers to offer rich features targeting microservice deployments [7–10], such as specialized container-based technologies to deploy and scale microservices, message brokers to allow for loose-coupled microservices, multi-tenant DBMS technologies to meet the isolation properties of microservices, and application frameworks and side-car technologies to speed up deployment, code evolution and maintenance of microservices.

Despite the benefits of the decoupled design, a recent study [41] demonstrates that practitioners encounter several challenges when trying to meet data management requirements in this architecture, including distributed transaction processing, data replication, consistent data and event querying and processing, and enforcing data integrity constraints. In short, microservices are designed to function independently, but in practice, they often rely on each other's data and functionality to complete a workflow. As a result, it is essential to benchmark microservices in a way that accurately reflects the needs of practitioners. Unfortunately, existing microservice benchmarks do not fully capture these real-world requirements [33, 41, 74]. For example, DeathStarBench [33] and TrainTicket [75] do not consider event processing, nor do they specify what data invariants and transactional guarantees are necessary. The absence of a comprehensive benchmark for measuring the performance of data management tasks in microservices leads to the development of custom benchmarks that fail to reflect the complexities of data management in real-world microservices [15].

To bridge this gap, we propose *Online Marketplace*, a novel microservice benchmark containing eight microservice types, ten event types, seven query types, including read-only and read-write queries, that reflect the key data management tasks pursued by practitioners, such as distributed transaction processing, data replication, consistent queries, event processing, and data constraint enforcement. To reflect the data management challenges mentioned above, we prescribe seven data management criteria that a data management platform should meet, including functional decomposition, resource isolation, data consistency, and data integrity of microservices. These criteria are meant to embrace the complex nature of deployments found in industry settings and facilitate conducting a fair comparison between different systems on the same basis. To our knowledge, *Online Marketplace* is the first microservice benchmark that embraces core data management requirements sought by microservice practitioners.

Based on the definition of *Online Marketplace*, we further developed a data generator and a benchmark driver. The latter can continuously generate and submit transactions to an implementation of *Online Marketplace*. A challenge of implementing the driver is generating transactions at runtime that coherently reflect the dynamic application state, for example, the latest product prices and product versions. Querying the runtime application state imposes an unnecessary and prohibitively expensive workload on the system. To address this problem, we developed a stateful driver, which manages a consistent mirror of some application data and generates coherent transaction inputs.

We verify the applicability of our benchmark by implementing five versions of *Online Marketplace* in three state-of-the-art data platforms, Orleans, Statefun, and a composite solution. Orleans and Statefun are designed for event-driven, distributed stateful applications. The composite solution reflects a usual architecture seek in practice [39, 41]. While implementing *Online Marketplace* on these platforms, we encountered several limitations that prevented us from fulfilling some of the data management functionalities and criteria sought in practice. We conducted experiments to measure how these competing platforms perform under different workloads and observed their design and architectural implications in performance. Our results show that our benchmark can effectively stress the performance of the platforms and reveal performance and functionality issues. Meanwhile, through *Online Marketplace*, we derive actionable insights to foment the design of

futuristic data systems that will meet the expectations of microservice practitioners, who are an important part of the database user community.

By providing a concrete example for database researchers to understand the unmet needs of data management in microservices, the benchmark serves as a testbed for experimenting with novel algorithms, approaches, techniques, and programming models. For instance, a recent study [42] aims to fulfill the data management criteria outlined here to enhance state management features in actor systems for cloud-native applications. Furthermore, Online Marketplace is a benchmark that not only benefits the real-world microservice applications that served as inspiration for our design [11, 19, 20, 22, 29, 30, 35, 38, 46, 48, 54, 56–58, 64, 68, 71] but can also extend beyond existing microservices applications. For instance, besides microservices, there are trends of other architectural styles and programming models offering trade-offs between complexities in data management and the degrees of coupling. These include modular monolithic architectures, FaaS, Actors, etc. As demonstrated in our case studies (§ 7), *Online Marketplace* is well positioned to serve as a testbed for these trending cloud application architectures and programming models. The artifacts used in this work are available online [40] for reproducibility and extension.

## 2 Background

**Data Management Challenges.** Despite more than a decade-long employment in the industry, little work in data management research has incorporated the challenges brought about by data-intensive applications following the microservices architectural style. Based on the findings of a recent study that empirically investigated the data management challenges in microservices [41] and public reports [11, 19, 20, 22, 29, 30, 35, 38, 46, 48, 54, 56–58, 64, 68, 71] of companies with large-scale deployments, we summarize the core data management challenges as follows:

(i) **Ensuring all-or-nothing atomicity.** The asynchronous and non-blocking nature of messages and the lack of interoperability across different data stores make distributed commit protocols difficult to implement, leading developers to either encode their own or eschew the use of synchronization mechanisms.

(ii) **Implementing efficient and consistent data processing.** While data is scattered across microservices, some workloads require querying and joining data belonging to different microservices. Therefore, developers often need to implement querying functionalities at the application layer that should belong in the database layer.

(iii) **Ensuring data replication correctness.** In order to reduce the expenses associated with querying data from remote microservices, developers often resort to caching or replicating data by subscribing to events generated by other microservices. However, as these events can arrive in any order, developers face challenges in maintaining consistent replication.

(iv) **Enforcing cross-microservice data integrity constraints.** As the application is functionally partitioned, data integrity constraints can span multiple microservices. This creates major challenges in constraint enforcement.

(v) **Ensuring correct event processing order.** Developers leverage application-generated events to implement event-based microservice workflows. However, guaranteeing the correct processing order can be challenging due to the asynchronous nature of events. Events can arrive out of order, late, or even duplicated. Such scenarios pose a significant issue when the application logic is sensitive to the processing order of events.

**Problems of Existing Benchmarks.** Existing microservice benchmarks like DeathStarBench [33] and TrainTicket [75] do not fully capture these real-world challenges, as shown in Table 1. DeathStar [33] was created to investigate the effects of microservice architectures on hardware and software in system stacks, particularly network and operating systems. TrainTicket focuses on replicating industrial faults in microservices, supporting researchers on investigating fault analysis

Table 1. Comparison of microservice benchmarks

| Requirement | Criteria | DeathStar & TrainTicket | Online Marketplace |
|---|---|---|---|
| Functional Decomposition | Isolation of Resources | Yes | Yes |
| Asynchronous Events | Event Processing Order and Delivery Guarantees | No | Yes |
| Event Processing | | No | Yes |
| Distributed Transactions | All-or-nothing Atomicity; Isolation Levels Allowed | No | Yes |
| Data Invariants | Data Invariant Enforcement | No | Yes |
| Data Replication | Data Caching and Replication Consistency | No | Yes |
| Query Processing | Query Processing Consistency | No | Yes |

and debugging. However, they do not take into consideration asynchronous events; thus, challenges related to correct event processing are missing. Furthermore, they do not consider any requirements on data invariants, transactional guarantees, data replication or data querying. Therefore, they target problems that are either oblivious to data management or position data management as not a primary concern.

Similar to microservices, Object-Oriented DBMSs (OODBMS) also adopt the modularization principle, such as encapsulating state and operations into objects within the database. Although, in principle, a microservice can be mapped to an object in OODBMS, we find that OODBMS benchmarks cannot meet the needs that we target. Benchmarks like Cattell, HyperModel, OO7, Justitia, and OCB aim for modeling object relationships in the context of engineering applications and evaluating object clustering algorithms [26]. All these fail to capture the data management requirements in Table 1.

TPC-W [24] is a transactional benchmark that models the core aspects of user experience on an e-commerce website, such as browsing pages, checking out books, and searching for keywords. TPC-C [25] was designed to reflect the transaction processing of a wholesale supplier. YCSB [23] models transactional workloads in the cloud that are not necessarily executed under ACID semantics. All of them assume a traditional monolithic application architecture. Our benchmark models a modern microservice-oriented application and the complex interplay of their components through events, differing substantially from these benchmarks regarding architectural style and data management requirements.

Unibench [73] offers OLTP and OLAP workloads for multi-model databases. However, it does not model the decomposition of microservices, which results in the absence of distributed and encapsulated states as found in microservice applications. On the other hand, stream processing benchmarks like Linear Road [13] focus on modeling continuous and historical queries, but they do not include transactional workloads.

Therefore, existing benchmarks cannot be used to benchmark the overhead of addressing the correlated data management challenges, which includes the overhead of all-or-nothing atomicity, concurrency control, consistent query processing, consistent data replication, and correct event processing. Consequently, they cannot be used to facilitate the development of new data systems to meet the real-world data management challenges of microservices.

# 3 The Online Marketplace Benchmark

## 3.1 Design Goals

To bridge the aforementioned gap, it is key to pick an application scenario that covers all the aforementioned challenges. We find online marketplaces meet the following design goals.

**Industry strength.** Online marketplaces present a substantial user base over the world [63]. It is widely reported that online marketplace platforms employ microservice architectures to achieve benefits such as loose-coupling, fault tolerance, workload isolation, higher data availability, scalability, and independent schema evolution [14, 17, 47, 69].

**Low reproducibility barrier.** In the same spirit of the design of TPC-C [25], online marketplaces fall into an application domain that can be easily understood by non-domain experts. This, in turn, reduces the learning curve and encourages benchmark adoption.

**Generalizability.** Many other popular application domains can be mapped to an online marketplace. These include multi-tenant e-commerce platforms, airline retailing [36] (e.g., hotels, travel agencies, and taxi and insurance companies as sellers), social media marketplaces, delivery service (e.g., connecting individuals to a myriad of businesses, like restaurants), digital investment (e.g., funds from different banks as products), and homestay platforms. It is noteworthy that many of these applications report data management challenges that we address in this work: event ordering and data invariants [19, 30, 54, 56–58, 71] transactional consistency across microservices [20, 29, 35, 38, 64], consistent data querying [22, 46, 48] data replication correctness [11, 54, 68].

In sum, an online marketplace is a suitable use case to benchmark data management in microservices.

## 3.2 Application Scenario

**Cart Management.** Customers shop in *Online Marketplace*by navigating and selecting products from a catalog. A customer session is linked to a *cart*, with operations involving adding, removing, and updating items (e.g., increasing the quantity). A customer can only have one active cart at a time. When requesting a checkout, the customer must include a payment option and a shipment address, which are assembled together with the cart items and submitted for processing. On the other hand, customers may also abandon their carts before submitting the checkout request.

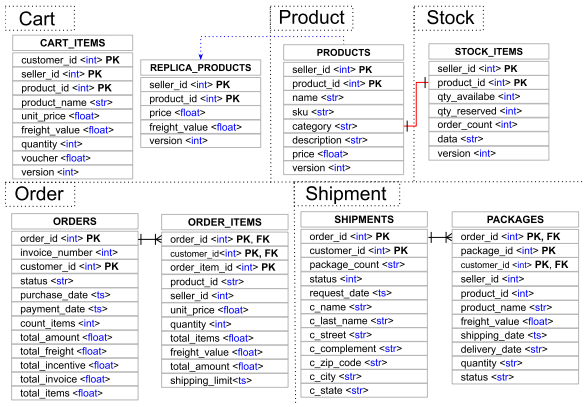
**Catalog Management.** Each product is offered by a particular seller. Sellers are responsible for managing their products and associated stock information. They may replenish products and adjust their prices.

**Customer Checkout.** Checkout requests follow a chain of actions: stock confirmation, order placement, payment processing, and shipment of items. Upon submission of an order, if one or more items do not have sufficient stock, the checkout will still proceed with the available items.

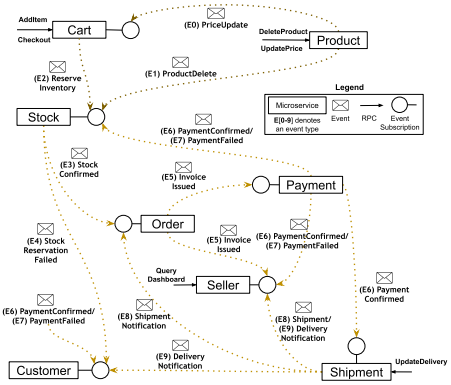
**Payment Processing.** Upon stock confirmation, the payment details provided by the customer are used to make a payment request to an external payment service provider (PSP) [37]. A payment can fail for two reasons: rejection of the transaction by the provider or impossibility of contacting the payment provider.

**Order Shipment.** Upon approval of payment, the shipment process starts. For each seller present in an order, a shipment request is created. A shipment request includes the items present in the order. Each item is reflected as a package that should be delivered to the customer at some point.

**Package Delivery.** Whenever a package is delivered, both the corresponding customer and seller are notified. When all packages of an order are delivered, the order is considered completed.




(a) Partial Data model




(b) Events

Fig. 1. Online Marketplace Microservices

### 3.3 Microservices

Traditionally, a microservice-based application is often made up of independent services, each deployed in a container on an OS-level virtualized platform such as Docker [41]. However, the emergence of programming frameworks for designing distributed applications, such as Orleans [16] and Statefun [62], provides programming models that allow microservices to be implemented using higher-level abstractions, such as actors and functions, respectively.

To allow this benchmark to support the gamut of microservice implementations, we withdraw ourselves from defining an architectural blueprint, but rather, we focus on describing the expected independent components that must compose the benchmark application. In this sense, in the following paragraphs, we take advantage of Figure 1 to clarify the interactions, APIs, and the data model prescribed for each microservice. It is worth noting that we use the term "events" as a communication abstraction for microservices, but these can also be framed as any message payload asynchronously delivered to a microservice. Besides, the event identifiers in Figure 1(b) do not imply a particular order in which the microservices exchange events. Furthermore, while we specify the microservices' state and the queries using the relational model, they can also be specified through other data and query models as long as the same functionalities can be achieved.

**Cart.** The *Cart* microservice allows the customer to manage the products that ought to compose a checkout order. It does so through APIs for managing cart items and submitting cart checkouts.

To manage a customer's cart, two relations are used: `carts` to track the status of a customer session, and *cart_items* to store the items that customers want to buy. Moreover, the *replica_products* relation represents a partial replica of the *product* table belonging to the `Product` microservice. The table is only updated through `ProductUpdate` events received from the `Product` microservice (represented by a dotted blue arrow in Figure 1(a)). The replication semantics are discussed in Section 4.3.

When a checkout request is received, `Cart` ensures the correctness of the customer's cart by matching the cart items to the replicated product information and looking for products that had their price updated. In this case, the price is updated and the difference (if positive) between the new and the old price becomes a discount in the cart item. The customer's cart items are then assembled into a `ReserveInventory` event and published for asynchronous processing. Once the event is published, the cart is sealed, and the customer is able to initiate a new cart session.

**Product.** The *Product* microservice manages the catalog of products. It performs operations over a single relation: `product`. For every update operation triggered via the following API, a corresponding event is generated for downstream processing: E0 for UpdatePrice and E1 for DeleteProduct.

*DeleteProduct*. Marks a product as unavailable to customers.

*UpdatePrice*. Update the price of a product.

**Stock.** The *Stock* microservice manages the inventory through a single relation: *stock_item*. To update the inventory, the *Stock* microservice processes four events: `ReserveInventory`, `PaymentConfirmed`, `PaymentFailed`, and `ProductDelete`. Every inbound event leads to updates in one or more *stock_item*' tuples (depending on the number of items present in the checkout).

On processing a `ProductDelete` event, the *Stock* microservice marks the respective *stock_item* as unavailable, meaning that future checkouts (triggered by `ReserveInventory` event) on this item cannot be carried out (represented by a red connector in Figure 1(a)).

Processing the `ReserveInventory` event leads to marking the stocks of some items (subject to availability) presented in the customer's cart as reserved. The result of processing `ReserveInventory` might lead to the generation of two events: (i) `StockConfirmed` if at least one product has been reserved, and; (ii) `ReserveStockFailed` if no product requested by the customer is available. In both cases, the respective items are assembled together in the respective event for downstream processing. Lastly, `PaymentConfirmed` confirms the reservation and `PaymentFailed` would lead to withdrawing the corresponding stock reservations.

Although at first sight, the differences between Product and Stock may seem blurry, it is important to highlight that they provide distinct functionalities in the application. While the `Product` is responsible for the correctness of the catalog data, including but not limited to the characteristics of a product like name, category, seller, and price, the `Stock` microservice ensures the integrity of the inventory. It is important to address that such design reflects real-world deployments [65].

**Order.** The *Order* microservice manages customer orders' data. It does so by maintaining four relations: *order*, *customer_orders*, *order_items*, and *order_history*. The *order* relation contains general data about an order, including but not limited to customer, amounts, etc. *customer_order* relation tracks the number of orders requested by each customer, which is used to form the invoice number. Order items relate to the products requested in checkout and *order_history* tracks updates to an order (invoiced, paid, shipped, completed). Order's relations are updated upon processing the following events: `StockConfirmed`, `ShipmentNotification`, `PaymentConfirmed`, `PaymentFailed`.

Upon receiving a `StockConfirmed` event, the amount to charge the customer is calculated (including freight and discounts) and an invoice number is generated, resulting in a `InvoiceIssued` event. During this processing, all four relations have tuples created (in case it is the first customer order, otherwise *customer_order* tuple is updated). Afterwards, there are two moments where a

customer's order status is updated: after a payment and after shipment updates. Both trigger writes to order and *order_history* relations. In this case, the order of event processing here plays a role on maintaining the notion of time progress for individual orders.

**Payment.** The *Payment* microservice is responsible for managing payment data. Two relations are used to track an orders' payment data: *payment* and *card_payment*. Payment processing starts through processing an `InvoiceIssued` event. Every credit applied to the order, namely, discounts and the payment option chosen by the customer at the time of the checkout (credit/debit card or a bank slip) are stored in *payment* relations and in case of a card payment, a *card_payment* tuple is also inserted.

As part of the payment processing, it may be necessary to coordinate with an external payment provider to ensure the provided payment method is valid. In this sense, *Payment* microservice relies on an idempotent API offered by the external payment provider. In case the payment microservice fails right after confirming a payment, subsequent attempts (after recovering from failure) to charge the customer does not incur in duplicate payments.

Processing an invoice can lead to two outbound events: `PaymentConfirmed` and `PaymentFailed`. The `PaymentConfirmed` event contains all items present in the paid invoice whereas the `PaymentFailed` is a simple payload containing the invoice number so the *Order* microservice can update its state accordingly.

**Shipment.** The *Shipment* microservice manages the lifecycle of a shipment processing, including shipment provision and delivery of goods. Two relations are maintained: *shipment* and *package*.

A shipment process starts by processing a `PaymentConfirmed` event, creating a delivery request for each order item and confirming all goods have been marked for shipment by producing a `ShipmentNotification` with status 'approved'.

At a later moment, through the *UpdateShipment* API, a set of shipments and associated packages tuples are updated. Each package delivered leads to the generation of a respective `DeliveryNotification` and, in case all items that form a shipment are delivered, a `ShipmentNotification` with status 'concluded' is also emitted.

**Customer**. The *Customer* microservice manages customer data, including their home address, contact information, and credit card. Besides, statistics about the customer are updated through processing the following events: `PaymentConfirmed`, `PaymentFailed`, `ReserveStockFailed`, and `DeliveryNotification`.

**Seller**. The *Seller* microservice manages seller-based marketplace data. It does so by processing events and transforming them into seller-centric data. Three relations are found in *Seller's* schema: *seller*, *order_entry*, and *order_entry_details*.

The relation *order_entry* represents an order item but from the perspective of a seller. In this sense, amounts related to the item (i.e., discount, freight, total) are calculated. Every order (derived from the `InvoiceIssued` payload) is transformed into *N order_entry* tuples, where N is the number of items corresponding to a seller. The relation *order_entry* is updated through processing the following events: `PaymentConfirmed`, `PaymentFailed`, `ShipmentNotification`, and `DeliveryNotification`.

### 3.4 Workload

In this section, we describe the *Online Marketplace*workload, namely, the business transactions and continuous queries the application must cope with.

*3.4.1 Business Transactions.* To realize the application scenario, we describe four business transactions that reflect different complexities in terms of the number of microservices involved and the number of events processed. In Section 4, we discuss how transactions possess different properties.

**Customer Checkout.** It starts in *Cart* microservice through processing a `Checkout` request. It involves *Cart*, *Stock*, *Order*, *Payment*, *Shipment*, *Seller*, and *Customer* microservices. A success path involves the following events: E2 - E3 - E5 - E6 - E8. There are two error cases: (i) when a payment fails, then from E5 we have E7 as the last; and when no single item from the cart is available in stock: E2 - E4. Although not shown in Figure 1 due to space constraints, a success path also involves E6 being processed by *Order*.
**Price Update.** To enable the partial replication of products in the *Cart*, upon processing a `UpdatePrice` request and updating its private state accordingly, the *Product* microservice generates E0 and sends it to the *Cart* microservice. By processing E0, the *Cart* microservice applies the new price to its corresponding replica.
**Product Delete.** To simulate making a product unavailable to customers, we pick a seller and a corresponding product (both from a distribution) and we set the product as disabled. To maintain the total number of products, thus avoiding anomalies in the distribution, we replace the deleted product with another one. This operation is done by processing a `DeleteProduct` request in *Product* microservice. Upon updating its private state accordingly, *Product* generates E1 and sends it to *Stock*.
**Update Delivery.** To simulate the delivery of goods, we pick the first 10 sellers with uncompleted orders (i.e., at least one package has not been delivered yet) in chronological order, and we set their respective oldest order's packages as delivered. Thus, packages are progressively delivered as more update delivery transactions are submitted to the system. For maintenance of statistics about customers and orders, the following events are generated with the transaction: `E9` is generated by *Shipment* for every package delivered and sent to the Seller and Customer microservices; and `E8` is generated by *Shipment* when all packages of a shipment are delivered and sent to *Order*.

*3.4.2 Continuous Queries.* Continuous queries over event streams constitute an emergent trend in microservice applications. As event payloads produced by different microservices often contain state information [32], it is possible to (indirectly) access data from multiple microservices without breaking their encapsulation. In other words, continuous queries can be built based on events without resorting to (synchronously) pulling data from each required microservice.

In this section, we describe three different types of continuous queries covering important concerns in *Online Marketplace*. To specify the queries, we use the syntax of Materialize [1], a streaming database. Due to space constraints, we explain one continuous query and the others can be found in our extended version [12].
**Query #1: Seller Dashboard.** The business scenario encountered in a marketplace requires sellers to have an end-to-end overview of the operation in real-time to identify trends, such as the popularity of products, and to support decision-making, such as when to increase product prices.
**Query Description:** In this query, we want to determine the total financial amount of ongoing orders by seller. Assuming there is a stream (Listing 1) that filters out concluded and failed orders, one could implement this continuous query as shown in Listing 2.

Listing 1. Ongoing order entries base query

```
CREATE MATERIALIZED VIEW order_entries
SELECT order_id, seller_id, product_id, ...
FROM InvoiceIssued as inv
LEFT JOIN ShipmentNotification as ship ON ship.order_id = inv.order_id
LEFT JOIN PaymentFailed as pay ON pay.order_id = inv.order_id
WHERE ship.status != 'concluded' AND pay.order_id IS NULL
```

Listing 2. Ongoing orders aggregation per seller

```
SELECT seller_id, COUNT(DISTINCT order_id) as count_orders, COUNT(DISTINCT seller_id, product_id)
     as count_items, SUM(total_amount) as total_amount, SUM(freight_value) as total_freight,
     SUM(total_items - total_amount) as total_incentive, SUM(total_invoice) as total_invoice,
     SUM(total_items) as total_items
FROM order_entries
GROUP BY seller_id
```

In addition, the seller dashboard output must also discriminate the records that compose the aggregated values computed in the above query. In this sense, it should also present to the user the following query result:

Listing 3. Ongoing orders discriminated

```
SELECT * FROM order_entries
WHERE seller_id == <sellerId>
```

**Query #2: Cart Abandonment.** A popular use case arising from a series of customer interactions is cart abandonment [44]. A cart is considered abandoned in two cases: (i) prior to checkout submission, and (ii) upon a failed payment processing, if no customer checkout re-submission is identified.
**Query Description:** In this query, we want to find the cart checkouts that have either failed via stock reservation or payment attempt and have not been involved in a new checkout attempt within the next 10 minutes after the failure. Upon detecting an abandoned cart, a `CartAbandoned` event is generated for both *Cart* and *Customer* microservices. The query specification can be found in our extended version [12].
**Query #3: Low Stock Warning.** To assist sellers, marketplaces usually monitor products' stock proactively to notify sellers about low inventory and ultimately refrain customers from experiencing unavailability of products.
**Query Description:** In this query, we want to find the products (and their respective sellers) that are likely to face unavailability in case no replenishment is provided in the near future. The search is based in the following criteria: The average number of items requested per week in the last month, independently of the result of the reservation and payment, is higher than the present stock level. The *Low Stock Warning* continuous query specification can also be found in our extended version [12].

## 4 Data Management Criteria

In this section, we discuss the criteria that an implementation of *Online Marketplace* should meet. The criteria reflect key principles and challenges of data management in microservices. Some criteria have multiple levels, allowing the benchmark users to choose the most suitable one that fits their specific requirements. Furthermore, our explicit criteria specifications facilitate conducting fair comparisons between different systems on the same basis.

### 4.1 Communication and Data Access

Microservices are functionally partitioned applications [53]. Two issues arise with this model:
(i) Microservices interact with each other through asynchronous events to carry out functionalities across multiple microservices. For example, completing a cart requires a composition of functionalities across *Cart*, *Stock*, *Order*, *Payment*, and *Shipment* through the events they exchange.
(ii) State management is divided into two categories: (a) Direct data access: Each microservice can directly access its own data (encapsulation principle); and (b) Indirect data access: Data within each

microservice can only be accessed externally through predefined interfaces (e.g., seller dashboard) or be notified about state changes through events (e.g., *Cart* is notified about product updates). It should be noted that this criterion does not require data from different microservices to be stored in different databases.

### 4.2 All-or-nothing Atomicity

The business transactions must comply with all-or-nothing atomicity semantics. This criterion is crucial to guard against crashes or performance degradation leading to failures in the middle of a business transaction.

### 4.3 Caching or Replication

Section 3.2 describes the case of *Cart* subscribing to product updates with the goal of ensuring that checkouts do not contain outdated product prices. As the strategy of implementation varies system by system, either cache or replication can fulfill this requirement. In this sense, we prescribe three possible correctness semantics:

(i) Eventual. Updates (price update or product delete) are processed independently, disregarding the order that they are generated at the source, i.e. *Product*.

(ii) Causality at the object level. Updates on the same product are processed sequentially in accordance with the order in which they were performed at the source.

(iii) Causality across multiple objects. All updates made by the same seller must be applied to the cart in the same order they were applied at the source, achieving read-your-write consistency.

### 4.4 State Safety Properties

*4.4.1 Inter-microservices Properties.* . Refers to a property that cut across microservices.

**S#1: Cross-microservice referential integrity.** *Stock* always references an existing product in *Product*. We take inspiration from the CAP theorem [34] to define two levels of consistency:

(i) Available System. A deleted product will eventually not be allowed to be reserved anymore. The transaction is considered committed after *Product* responds to the `Delete Product` request.

At this level, if there is a network error or failure of the microservices, the delete message (EY) may not reach *Stock*. That requires the inconsistency to be detected and resolved at a later point.

(ii) Consistent System. `Delete Product` request is committed only when both Product and Stock have committed, therefore Product and Stock are always consistent with each other. At this level, a deleted product will not be available for future checkout attempts.

**S#2: No duplicate checkouts.** The cart of a customer session must not be checked out more than once. This can be safeguarded by having mutual exclusion in each cart. Another way to ensure this property is by assigning a customer session ID to each new customer session. The ID is included in the payload of each checkout request submitted to *Order*. Upon receiving it, *Order* ensures the same cart checkout does not lead to duplicate order processing.

*4.4.2 Intra-microservice Properties.* These include properties that can be enforced by relying on only local states. They can be achieved through appropriate isolation levels.

**S#3: No overselling of items.** In *Stock* microservice, both available and reserved quantities cannot fall below zero for every item. In addition, the reserved quantity must never be higher than the available quantity.

**S#4: Maintenance of customer statistics.** The system tracks successful and failed payments and deliveries through increments of numbers. An appropriate isolation level is required to ensure that the increment in *Customer* is not missed.

**S#5: Linearized product updates.** Updates on each individual product's price or version in the *Product* state must be linearized.
**S#6: Concurrent Update Delivery transactions must execute in isolation.** Since several events can be generated during a *Update Delivery* transaction, it is essential that concurrent *Update Delivery* transactions do not operate on the records of the same shipments and packages. The purpose is to avoid emitting duplicated `ShipmentNotification` and `DeliveryNotification` events.
**S#7: Stock operations triggered by events must execute in isolation.** Processing events in *Stock* requires either serializable isolation or exclusive locks on the items contained in the event payload.

### 4.5 Event Processing Properties

**Event Order.** Event processing order impacts *Order* and *Seller* dashboard. A microservice data platform can provide two levels of event ordering guarantee.
Unordered: `InvoiceIssued`, `PaymentConfirmed` and `ShipmentNotification` events are processed arbitrarily, possibly leading to violating the natural order processing workflow.
Causally Ordered: `InvoiceIssued` must precede payment events. The `PaymentConfirmed` event must always precede shipment events. Similarly, a `ShipmentNotification` with the status 'approved' and `DeliveryNotification` must always precede the corresponding `ShipmentNotification` with the status 'concluded'.
**Consistent Snapshot for Continuous Queries.** For the two concurrent queries of the seller dashboard to be consistent with each other, their results should reflect the same snapshot of the application state.
**Event Delivery.** The data platform can provide three levels of event delivery guarantees, namely at-most-once, at-least-once, and exactly-once delivery. The delivery guarantee has impacts on how to implement continuous queries and business transactions in *Online Marketplace* to achieve correctness. At-least-once and at-most-once delivery can make queries inaccurate. For example, while at-most-once delivery provides a lower bound on the profits in the *Seller Dashboard* without replaying lost messages, at-least-once can provide skewed results without accounting for duplicate events.

For transactions, at-least-once delivery requires microservices to be idempotent to account for possible duplicate event processing. On the other hand, a timeout mechanism must abort transactions if at-most-once or exactly-once delivery is used to prevent stalled transactions from blocking the other transactions.

### 4.6 Performance and Failure Isolation

The functional partitioning of the application allows microservices to operate independently, offering benefits from the isolation of resources assigned to each microservice [39, 53]. Implementations of *Online Marketplace* can achieve isolation in two tiers:

(i) Each microservice's code/logic is executed on different computational resources, achieving performance and fault isolation at the application tier. This minimizes the impact of resource usage and failures between different microservices.

(ii) In the database tier, database operations do not interfere with each other in terms of performance and failure, achieved through isolated resource allocation to databases.

### 4.7 Logging

Audit logging is a critical concern in applications as it allows developers to track application events like user activities (e.g., checking out a cart) and state updates [27]. These events recorded during application execution serves as an audit trail, aiding developers in troubleshooting faults and verifying compliance with prescribed business rules. This is even more pressing in distributed

systems, such as microservices. The substantial exchange of events and the complex interplay of independent components make it challenging to reason about errors and failures involving multiple asynchronous microservices. Thus, data systems must store audit events durably. In *Online Marketplace*, there are two key events related to logging historical records of operations: `ShipmentNotification` and `PaymentFailed`.

**(a)** Upon `ShipmentNotification` with `status` 'completed' or `PaymentFailed`, *Order* logs all records associated with such an order, in particular the relations *order*, *order_items*, and *order_history*.

**(b)** Upon `ShipmentNotification` with `status` 'completed', *Seller* logs all records associated with such shipment, in particular the relations *order_entry* and *order_entry_details*.

**(c)** As part of the emission of a `ShipmentNotification` with `status` 'completed', *Shipment* logs all records associated with such shipment, in particular those in the relations *shipment* and *package*.

**(d)** As part of payment processing, *Payment* logs payment records independently of the outcome (success or failure), particularly the relations *payments* and *payment_cards*.

## 5 Data and Workload Composition

The workload submitted to *Online Marketplace* can be adapted to fit particular needs. In the following, we present the different configuration parameters that can de defined for experiments.

**Data Population.** The state of some of the microservices require initialization prior to workload submission. The following procedure is expected, where $X$, $Y$, and $Z$ are configuration parameters:
(i) $X$ number of customers are inserted into *Customer* microservice;
(ii) $Y$ number of products are inserted into *Product* microservice;
(iii) $Y$ number of stock items are inserted into *Stock* microservice. Each *stock_item* tuple must refer to an existing product (through *seller_id* and *product_id* columns) in *Product* microservice;
(iv) $Z$ number of products per seller leads to $(Y/Z)$ seller tuples inserted into *Seller* microservice. In case of positive remainder, the last seller must own W products, where $W < Z$.

On the one hand, the number of customers should be large enough to accommodate the maximum amount of concurrent transactions running the system at a given time. For instance, considering a throughput $T$, $X$ must be higher than $T$ to guarantee there is always a customer available. On the other hand, the number is limited by the amount of memory available.

Furthermore, the number of products and sellers should be large enough not to create many conflicts when running. Considering *prob_s* and *prob_p* as the probabilities of picking a seller and of picking a seller's product, respectively, to have less than $\delta$ conflicts, one should pick distributions that lead to $T * prob_s * prob_p < \delta$.

To aid data population, the dataset can be generated based on a given *size factor* following an uniform distribution. A size factor $S$ leads to 10K customers ($X$), products and also associated stock items ($Y$). Benchmark users can populate records in other microservices to simulate preexisting data and introduce a degree of overhead in state accesses during the workload execution. A size factor $S$ leads the *order* table to be initialized with $S$*100K tuples uniformly distributed among customers, having the number of *order item*s randomly picked (1-10). In consequence, *payment*, and *shipment* follow the same size given their tuples refer to an existing order.

**Workload Distributions.** Distribution in the workload is centered on sellers and their products. For every operation involving a product (add cart item, product price update, and product delete), one has to pick first a seller and then proceed to pick a corresponding item from the seller's product keyset. In this way, Uniform and Zipfian distributions can be used interchangeably in two cases:
- Seller selection: for every product selection, one has to first pick a seller based on a defined distribution.
- Product selection: After a seller is previously selected, one picks a seller's product based on a defined distribution.

**Metrics.** We collect two metrics in this benchmark. Throughput is the number of transactions processed per second, which includes both business transactions that have been successfully completed and continuous queries that have successfully returned a result. End-to-end latency is measured from the moment a client sends a transaction request until the result is received.

**Transaction ratios** define the probabilities of clients submitting various types of business transactions and continuous queries to the system. We envision it can be set to reflect some potential scenarios:

**A. Order-heavy scenarios** reflect e-commerce scenarios and high-sales periods, the workload is dominated by order processing (i.e., checkouts and deliveries), having a small percentage of abandoned carts and payments declined. This workload includes both uniform and non-uniform seller distributions, which models for example, some sellers' products being much more popular than others, leading to orders being canceled due to unavailability. The parameters are inspired by Olist workload [5] and specified as product delete with 2%, price update with 3%, seller dashboard with 5%, customer checkout with 30%, and update delivery with 60%.

**B. Update-heavy scenarios** reflect frequent updates with low-latency requirements, as found in banking, financial trading, and airline applications. This workload models a substantial degree of product updates with non-uniform seller and product distributions. Varying the zipfian constants allows for controlling the number of accesses to Cart and Stock, similar to some extended versions of Smallbank [43, 55]. One can set the zipfian constants ranging from 0.2 to 1.2, and transaction ratio as product delete and price update with 30% each, seller dashboard with 2.5%, customer checkout with 12.5%, and update delivery with 25%.

**C. Hybrid transactional-analytical scenarios** model a substantial number of users continuously querying their data (i.e., the seller dashboard) while the system must also cope with moderate transactional requests. The goal is to prevent the use of simplified caching mechanisms. Thus, seller distribution is uniform while products may have skewed access. Transaction ratios are inspired by HTAP benchmarks [21, 72] and defined as seller dashboard with 48%, customer checkout with 16%, update delivery with 32%, product delete and update price with 2% each.

## 6 Benchmark Driver

### 6.1 Driver Functionalities

To manage the life-cycle of an experiment composed by: (i) data generation, (ii) data ingestion, (iii) data check, (iv) workload submission, (v) collection of results, (vi) report generation, and (vii) cleaning of states, we develop a benchmark driver in .NET [12].

A user specifies the data population, workload composition, ratio of transactions, and distributions through a configuration file. A user usually starts with generating data. Sellers, customers, products, and stock items are generated using Bogus [18], a popular library for generating synthetic but realistic data, including email, full name, address, and social security number. For products, Bogus generates matching product names and descriptions, including prices and locations. Generated data can be either stored durably or kept in main memory via DuckDB [6]. Users can load the data from DuckDB and move on to data ingestion, populating the microservices with initial data. Further details of the driver software, including workload generation, can be found in the extended version of this paper [12].

### 6.2 Driver Implementation Challenges

The *Online Marketplace* benchmark presents particular characteristics that necessitate caution on submitting transactions.

**Simulating Customer Sessions.** We start with the *Cart* microservice, which can be exemplified as a stateful operator. In the context of a customer session, a cart state evolves by having sequential operations (i.e., add cart item) up to a point where the state is sealed (through a checkout request), then returning to the initial state. To this end, the workload submission must account for each customer session individually. In case there is an active customer session for customer X, *there should be no concurrent driver's thread simulating the same customer.* The reason is that concurrent operations on X's cart state can interleave, making it difficult to maintain the workload distribution.

To meet this requirement, a thread-safe queue for idle customers is maintained. Whenever a thread initiates a checkout transaction, it picks a customer from the idle queue to represent. After the checkout request is submitted, the thread pushes the customer to the queue, making it available again for other threads. The time a customer lies in the idle queue is driven by the concurrency level supported by the target platform. The faster checkouts are processed, the less time customers spend in the idle queue.

**Managing Coherent Product Versions.** As explained in Section 3.4.1, products could be deleted and replaced by new ones. In other words, new product versions are being generated online continuously. It is crucial to generate transactions referring to coherent product versions. For example, to generate a price update transaction, we need to make sure to refer to the current product version in *Product*'s state. As another example, on adding an item to a cart, we need to make sure cart items refer either to the current product version in *Product* or the version that preceded the current one at the time of this operation. The goal is to simulate the latest product version "seen" by a customer.

To obtain a coherent product version while constructing transactions, we have to know the application state. However, the benchmark driver should not query microservice states during transaction submission for two reasons: (i) Workload generation should be independent of the actual data platforms being used; (ii) Querying the state of microservices would introduce additional load that is not prescribed in the benchmark.

In order to generate correct transactions while not querying the microservices' states, the driver manages internally a consistent mirror of the *Product* microservice state to guarantee access to coherent product versions. The driver linearizes the submission of concurrent update requests to the same product; the compare-and-swap mechanism is used to decrease synchronization costs.

Whereas the picking of product versions for building a cart item runs concurrently with product updates so that customer and seller threads do not block each other.

**Matching Transaction Requests to Asynchronous Results.** It is common that platforms for building microservice applications provide results asynchronously (Section 6.1). In such an asynchronous system, the driver must track each submitted transaction and match it with the corresponding asynchronous transaction result to compute the metrics accurately. To this end, each transaction request is assigned a timestamp and a unique ID. This ID is later used to match the request to a corresponding transaction result that is eventually received. To avoid threads blocking each other, ID generation is decoupled from transaction submission. In conclusion, a stateful driver is necessary to provide correct transaction input.

## 7 Case Studies

To demonstrate the applicability of *Online Marketplace* and show the design implications and effects on different systems, we implement and experiment five versions of *Online Marketplace* in three competing platforms, Orleans, Statefun, and a custom solution. Orleans and Statefun are designed to develop event-driven services with state management functionalities. The custom solution is based on a combination of multiple systems that reflects an usual architecture seek in practice [39, 41], which will also provide an insight into the complexity faced by the developers.

From the implementations and experimental study, we derive lessons learned (referenced by **L#**) and design decisions (**D#** in § 7.2.3) to foment new systems' design and improve both state-of-the-art platforms, highlighting the usefulness of *Online Marketplace*.

## 7.1 Implementations

*7.1.1 Microsoft Orleans.* Orleans is a framework for building distributed stateful applications. In Orleans, applications are composed of concurrent virtual actors [16], each encapsulating a private state, that are allocated transparently across machines [3]. Developers are offered APIs to log actor state [4], which allows actors to recover their state upon crashes or server migrations. In our implementation, we use the default Orleans actors' memory storage API to store the microservices data.

Due to the single-threaded actor abstraction, developers are encouraged to decompose application functionalities into distinct actors to avoid a few specific virtual actors becoming the bottleneck. Inspired by the guidelines of Wang et al. [70], our design aims to maximize parallelism and minimize transaction latency by assigning *Online Marketplace* functionalities to different actors. Due to space constraints, further details about the components' design is found in our extended version [12]. We implement *Online Marketplace* on Orleans 7.2.1, using both the default non-transactional Orleans API and the transactional API, referred to Orleans Transactions (TX) in the rest of the paper. To allow Orleans actors to be reachable from the driver, we deploy an HTTP server on top of the Orleans silo. The server is responsible for parsing and forwarding incoming requests to the appropriate actors, reporting back the transaction results via an HTTP interface.

*7.1.2 Flink Statefun.* Statefun is a platform built on top of Flink for running distributed applications based on the concept of stateful functions [62]. Flink processes manage the state, handle incoming requests, and invoke the appropriate functions [60]. Each function encapsulates its own logical state and reacts to incoming messages asynchronously. Each incoming message is processed sequentially, in a way similar to Orleans actors. As a result, applications can be designed by composing functions through messages. Similarly to Orleans, we use the Statefun memory storage API to store the microservices data.

We implement *Online Marketplace* on Statefun 3.3, and, given the resemblance of both programming models, we opted to model stateful functions with the same design as Orleans. To match the architectural design found in web services, we deploy Statefun with an HTTP ingress. Once the driver submits the transaction input, the ingress acknowledges the reception of the request and dispatches it to the appropriate function. We also deploy an HTTP egress to allow for the collection of transaction results that are eventually completed. The function that terminates a transaction sends a completion message to the egress. The egress operator stores the result and makes it available to clients. This design follows the Statefun documentation [61]. To execute Stateful Functions runtime, we follow the recommended deployment mode using docker images and the most performant execution style (co-located functions) [59].

*7.1.3 Implementation Insights.* Upon implementing the *Online Marketplace* features in each platform, we encountered some limitations. We explain in the following how we mitigated them and the criteria that the target platforms can meet. Note that we only consider features that are natively supported by the framework, and use as few external systems as possible.

**Data replication.** Given that both platforms do not provide indexing for the actor or function states, we cannot efficiently query which carts contain a particular product. Although we could carry out this search iterating over all carts of the keyspace, we found this incurs a heavy cost operation, significantly impacting the benchmark analysis. Therefore, we opted not to implement this replication feature in the platforms.

**Continuous queries.** We do not use external stream processing systems in order to benchmark only the target platforms rather than the integration of multiple systems. Therefore, we opted to only implement the query dashboard, but not the continuous queries of cart abandonment and low stock warning. The reason is that the `order_entries` view can be maintained through conventional data structures and updated through application events, while the others require more complicated stream processing operators such as windowed aggregates and joins, which are not provided natively by the two platforms.

**L#1.** *Apart from being complex and error-prone, ad-hoc replication and continuous query implementations incur an overhead on both Orleans and Statefun platforms.*

**Messaging delivery guarantees.** Orleans provides an at-most-once delivery guarantee by default. Although Orleans can be configured to send retries upon timeout, we opted to capture the timeout exception and report to the driver that the transaction has been completed with an error. The reason is that, by enabling retries, the message may arrive multiple times, potentially corrupting data if the application is not idempotent [53].

On the other hand, Statefun manages state storage and message delivery in an integrated manner, such that in case of a delivery error, Statefun transparently retries the delivery up to a timeout and, upon that, rewinds the application to a previously consistent checkpoint [28]. To make the performance results of the two platforms comparable, we disabled checkpointing in Statefun, and in case of a delivery error, we proceeded in the same way as in Orleans.

**L#2.** *The lack of exactly-once processing guarantees in Orleans requires developers to make their operations idempotent, adding execution overhead and increasing implementation complexity.*

**Logging.** We do not use Orleans storage because we run into the problem of inconsistent state while logging the actor's state [2]. Instead, we use external PostgreSQL to log completed transactions on both platforms. The consistent mechanisms also make the results of the two platforms comparable.

**Resource Isolation.** Both actors in Orleans and functions in Statefun share computational and data storage resources of the node they are allocated to. To isolate the computational resources of a specific microservice, it is necessary to configure a custom grain placement strategy in Orleans [50]. In Statefun, functions are allocated to Flink Task Managers transparently [31]. To make the results comparable, we use Orleans default placement strategy.

**L#3.** *The lack of explicit interfaces for configuring function allocation in Statefun can jeopardize achieving performance and failure isolation for microservices.*

*7.1.4 A Composite Solution.* Our experience above matches recent findings that microservice practitioners must combine several heterogeneous systems to fulfill their data management requirements [41]. Inspired by these results, we use a popular stack of technologies that practitioners often use to design a full-featured *Online Marketplace* implementation.

To achieve all-or-nothing atomicity and concurrency control, we base our solution on Orleans Transactions. We offload consistent querying to PostgreSQL. With a combination of materialized view and refresh view triggered via actors , we achieve a consistent seller dashboard snapshot through a PostgreSQL transaction.

We implement two replication semantics: (i) Eventual, enabled by Orleans Streams [52], an Orleans add-on that allows actors to subscribe and publish to streams dynamically; and (ii) Causality across multiple objects, enabled by a primary-secondary Redis deployment. To decouple writers from readers, we use the following scheme: *Product* actors write to the primary node, and updates are asynchronously streamed to the secondary node, so *Cart* actors can read from it. As Redis linearizes writes, updates are applied to the *Cart* in order.

**L#4.** *Achieving consistent replication and querying requires weaving together heterogeneous systems through non-native APIs.*

## 7.2 Experimental Study

*7.2.1 Experimental Settings.* **Deployment.** We set up our benchmarking environment on UCloud [67] based on u1-standard instances. A u1-standard contains an Intel Xeon Gold 6130 CPU@2.10 GHz, 32 CPUs, and 384 GB of memory [66]. UCloud instances run inside a Kubernetes cluster connected via 100Gbps Infiniband virtual network. To avoid resource competition, we allocate independent instances to different benchmarking components, namely, the benchmark driver, the target platform, the PostgreSQL database server, and the Redis instances. To remove cache effects, we restart Orleans and Statefun after each run. In experiments involving Redis and PostgreSQL, we also clean up the state after each run. Besides, after data population, we ensure CPU usage returns to idle before initializing transaction submission. We use PostgreSQL 14.5 running in the operating system (OS) Debian 12.1, whereas the driver and platform instances run in the OS Ubuntu 22.10. All the instances are located in the same region and availability zone. The criteria in §4.6 are only applied to multi-node deployments. Since we focused our investigation on the performance and overhead of the platforms in multi-core settings, these are not applicable.
**Methodology.** All experiments are run in 6 epochs of 10 seconds each with the first 2 epochs as a warm-up period. We collect two metrics: throughput and end-to-end latency. For each experiment, we maximized the resource allocated to the instances running the benchmark driver and PostgreSQL and we made sure resource usage was kept under 80%, to avoid them becoming the bottleneck.
**Workload and Parameters.** We use the workload modeling **scenario A** (§ 5). We set the probability of payments accepted and checkout to 100% to maximize the amount of transactions. Checkouts contain up to 10 items and quantities are selected randomly for each item (1-5). Each cart item has 5% probability of being assigned a discount. The discount value is randomly picked, not surpassing 10% of the item's price. We use both uniform and skewed distribution for picking sellers per each transaction. The number of sellers and products are picked reflecting *size factor* 10, so to not introduce substantial conflicts in uniform distribution.

*7.2.2 Experiment Results.* **Driver Scalability.** To examine if the coordination mechanisms of the driver (Section 6.2) are scalable, we conduct a scalability experiment. We use the parameters of **scenario A** (§ 5) and fix the transaction latency to 100 ms, varying the number of concurrent workers submitting transactions. We observe in Figure 2(a) that the number of completed transactions scales with the concurrency level, showing stable throughput across epochs. The overhead entailed by adding more worker threads does not create a bottleneck, independent of the distribution picked for sellers and products, reflecting the efficiency of the driver's coordination mechanisms.
**Effect of Concurrency Level.** In this experiment, we refer to *concurrency level* the maximum number of concurrent transactions running in the system at a given time. We measure the overhead incurred by different concurrency levels and identify the parameter that provides the optimal performance in each platform for further evaluation. In this section, we maximize the resources available (32 CPUs) and we set the workload skewness to be uniform for both sellers and products.

As shown in Figure 2(b), Orleans achieves maximum throughput with the concurrency level 48. With an even higher concurrency level, more messages would be accumulated on actors' input queue, particularly actors in the checkout critical path (e.g., with *Order* and *Payment* doubling their processing latency). The overhead introduced by logging is negligible at the start and remains constant as the concurrency level increases, achieving a maximum of 12% overhead compared to non-logging. The lower throughput derives from the wait introduced by updates submitted to PostgreSQL.

As for Statefun (Figure 2(c)), in order to achieve maximum throughput, the concurrency level required is significantly higher compared to Orleans. We conjecture several reasons: (i) Although we aimed for the most performant deployment style, the containerized deployment mode introduces

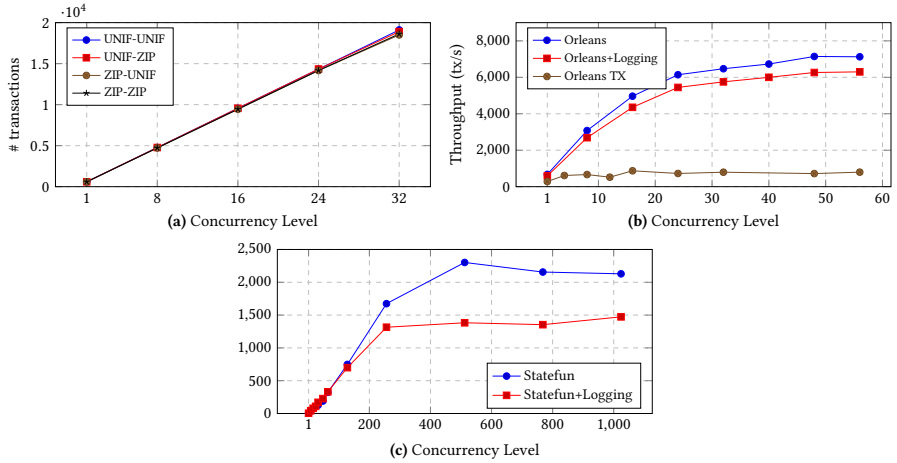



Fig. 2. Driver Scalability (a) and Concurrency Overhead (b & c)

overhead on execution functions due to the virtualization layer; [1] (ii) The HTTP ingress acts as a single operator, processing requests serially, introducing a bottleneck; [2] (iii) Although we made sure to adjust the polling rate to a configuration that maximizes Statefun performance, the polling for transaction results coming from the driver to compute metrics in a timely manner invariably introduces a processing overhead.

Although negligible initially, logging shows a steep increase from the 200 concurrency level, maintaining a stable overhead afterward. The difference in overhead from Orleans lies in the benefits of reentrancy [49] found in virtual actors, which minimizes the effect of waiting for responses from external systems.

**L#5.** *The inherent deployment and processing model of the dataflow architecture of Statefun are key factors that harm performance in transactional and event-based workloads.*

**Effect of Workload Skew.** A skewed workload causes the access of certain records to become more frequent over time. Therefore, it increases contention on a small subset of actors/functions. While we measured the platforms under a low skew level in the previous experiments, in this one, we use the `zipfian` function API in the MathNet library [45] to generate different skewed workloads. To this end, we select different `zipfian` constants to vary the degree of contention in the workload.

Preliminary experiments showed that fixing seller distribution and varying product skewness showed similar throughput across different skew levels, which led us to investigate the effects of seller skewness. Thus, we picked the sellers using a Zipfian distribution while picking products using a uniform distribution. Again, we use 32 CPUs and the concurrency level that maximizes throughput. Figure 3(c) exhibits the Zipfian value of six skew levels used. [3]

It is observed that the throughput of Orleans decreases with increasing skewness, following a stable trend. This phenomenon is expected since there is a high contention on certain sellers and their products. Statefun, on the other hand, is less sensitive to workload skewness. In this case, we conjecture that the effects of batching messages play a role, allowing the scheduling of functions not to incur overhead as skewness increases.

[1]We could not confirm this overhead because we do not find instructions in the documentation to run Statefun in bare metal.

[2]We tried using RichParallelSourceFunction to enable parallel ingress, but we found that driver requests were arriving out of order in functions, impeding the execution of some transactions.

[3]Statefun crashes on skew levels 1 and 1.2. The metrics are captured up to the crash.

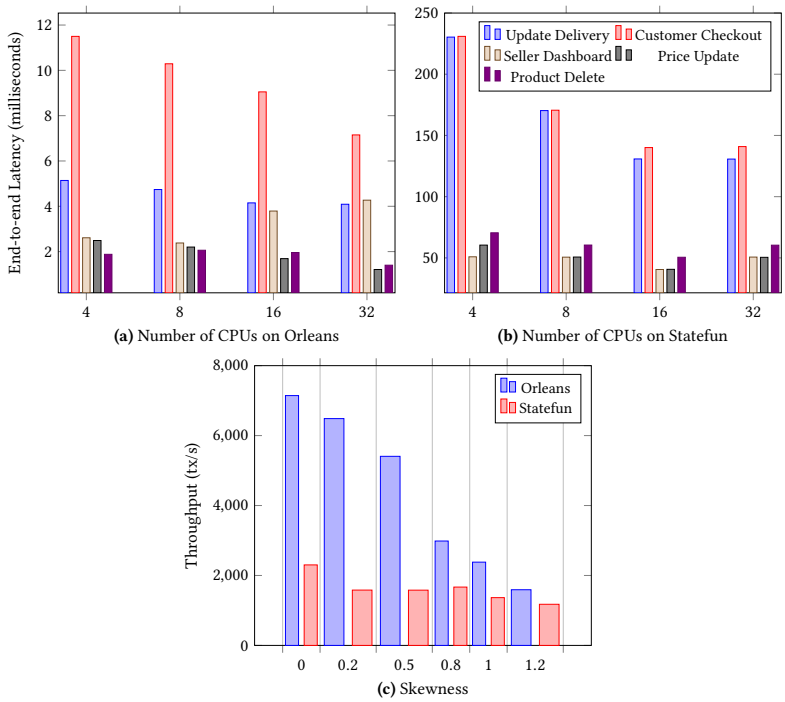



Fig. 3. Breakdown Latency (a & b) and Workload Skewness (c)

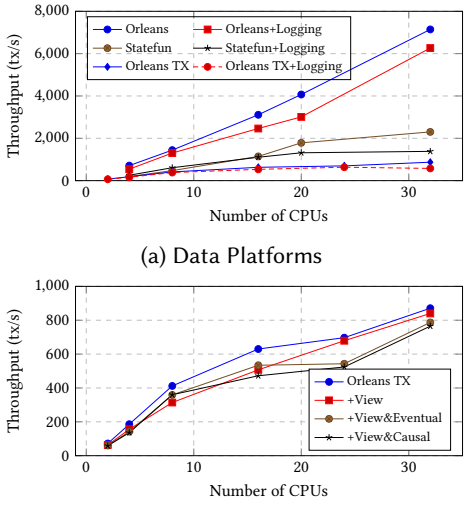



Fig. 4. Scalability

**L#6.** *A skewed execution introduces a lower performance penalty in Statefun given the inherent batch-oriented execution model.*

**Scalability.** In this experiment, we evaluate the scalability of both platforms by measuring performance metrics as more resources become available. We do so by increasing the number of cores from 4 to 32 and varying the concurrency level accordingly.

Figure 4(a) shows Orleans scales linearly as we increase the number of CPUs. In a similar way, Orleans with logging scales nearly linearly. The effects of logging remain low at the start, slightly

increasing from 16 CPUs on and remaining stable afterward, matching the expected overhead found in concurrency experiments (Figure 2(b)). Statefun can also take advantage of increased computational resources but to a lower degree compared to Orleans. The overhead of logging is nonexistent at the start, and from 16 CPUs on, it impacts Statefun scalability.

On the other hand, we found Orleans Transactions add a significant overhead by enforcing all-or-nothing atomicity and concurrency control. Even runs with a lower concurrency level are dominated by queued lock requests, often leading to message timeouts, thus affecting throughput significantly. We attempted to tune the timeouts of message and lock requests, but no configuration led to improvements over the default one.

**L#7.** *Ensuring all-or-nothing atomicity and concurrency control appear as dominant overheads on both platforms.*

Figure 3 shows the latency breakdown of the transactions. In line with the throughput, customer checkout, and price update transactions, end-to-end latency in Orleans decreases as more CPUs are available. On the other hand, the seller dashboard shows increasing latency because the more CPUs available, the more concurrent transactions exist in the system, which introduces more customer orders, thus increasing the processing time to compute the query result. Update delivery transactions are not affected by the same phenomena because the processing is independent of the number of orders in progress in the system.

As for Statefun, for customer checkout and update delivery, it is possible to observe latency significantly decreases as more CPUs are available, showing improvements up to 61% from 4 to 16-32. On the other hand, the overall throughput results are reflected in the longer end-to-end latency. One of the aspects that drives Statefun's overall latency higher compared to Orleans is inherent to the programming model. Whenever an operation involves interacting with multiple functions, a requester function must send a message to each recipient function and wait for the eventual arrival of the responses. The function must keep track of the responses received through custom-made code, which necessarily involves storing responses in the function's state. On the other hand, in Orleans, multiple actors' calls are encapsulated through promises and do not involve state operations. This scenario occurs in both checkout and update delivery transactions to meet properties 1, 3, 6, and 7.

**L#8.** *The absence of native primitives for coordinating asynchronous operations across multiple functions significantly impacts Statefun's overall performance.*

Turning our attention to the composite solution, Figure 4(b) shows the overhead of consistent query (+View) and different replication semantics (Eventual and Causal). The results show that the overhead introduced by causal replication is not substantial compared to eventual, which can motivate users to overlook weak semantics in their deployments. On the other hand, the overhead of replication tends to increase as more transactions compete for resources. In the eventual case, we conjecture that, as Orleans Streams are also implemented as actors [51], they compete with transactional actors for scheduling turns. Similarly, in the causal case, the cost of the increasing number of network I/O operations tends to overcome the benefits of memory-resident data provided by Redis. We omit the results with logging because all run with an average overhead of 5 to 10%. It is worth noting that we managed to implement the missing criteria adding less than 15% overhead on average, **demonstrating that meeting all the benchmark criteria is realistic**. However, the composite solution performance is bounded by Orleans Transactions.

**L#9.** *Offloading replication and continuous queries to external systems inevitably leads to increased network I/O overhead.*

*7.2.3 Design Decisions.* In this section, based on the observations and lessons learned identified through our case studies, we devise a set of design decisions to advance data platforms and foment new ones.

In the **feature** space, we posit the following to address **L#1**, **L#4**, and **L#9**:
**D#1.** *Enhancing microservice platforms with native, system-level support for data replication and querying microservice states is a promising feature.* This must be paired with performant system-level implementations. The inherent design of existing data platforms may limit the embracement of these missing features, leading to the next two design decisions.
**D#2.** *Integrating external systems and their features into microservice platforms is another potential candidate to relieve the burden on developers.* The case study showed it is realistic to rely on features provided by external systems. However, fulfilling missing requirements must not jeopardize performance.
**D#3.** *Integrating the features from external systems must be accompanied by optimizations to hide latencies associated with cross-system coordination to prevent substantial performance penalty.*

In terms of **performance**, lessons **L#5** and **L#8** demonstrate the values of the following design decisions:
**D#4.** *The event processing model and task scheduling in microservice platforms should reduce the impact of the long latency of certain multi-microservice workflows (e.g., customer checkout) on shorter transactions (e.g., price updates).* These facilities should ideally be transparent to developers, as in Orleans. In addition, designing primitives to circumvent concurrency limitations inherent to the programming model is another opportunity, as follows.
**D#5.** *Multiplexing multiple non-conflicting transactions within a scheduling unit (e.g., enabling grain reentrancy in Orleans) is a promising technique to speed up microservice workloads,* enhancing throughput by running non-conflicting transactions concurrently.

From a **system architecture** perspective, lessons **L#2** and **L#6** derive the following:
**D#6.** *Holistically managing state updates and message queue operations found in Statefun appears to be a promising approach for ensuring exactly-once semantics.* That relieves developers from writing idempotent code to prevent duplicate executions, which may not always be possible. These should be paired with efficient solutions to schedule application functions.
**D#7.** *Batching multiple conflicting transactions is an effective performance technique for highly-contended microservice workloads.* Statefun inherits the operator execution model of Flink, which processes event streams in batches. Orleans, on the other hand, tracks message timeouts individually, which limits batching opportunities.

Lesson **L#7** derives better **algorithmic design** and lesson **L#3** addresses the usefulness of proper **abstractions**, respectively:
**D#8.** *Devising new algorithms and techniques for achieving all-or-nothing atomicity and concurrency control is critical for microservice performance.* Recent research shows it is possible to speed up Orleans Transactions by 4X through deterministic scheduling [43]. Offering weaker isolation levels that can still achieve the required properties is another promising direction. For instance, customer statistics can be updated in any order as long as the events are processed exactly once.
**D#9.** *Virtualized representation of application functions and associated states, such as through a virtual actor in Orleans, appears as an effective abstraction* to manage deployment, discovery, distribution, scheduling, and failures holistically in microservices.

## 8 Concluding Remarks

Our experiments with five implementations of *Online Marketplace* in three competing systems have shown varied results under different workloads, showing the ability of *Online Marketplace* to pinpoint missing core data management features and the impact of the programming abstractions and architectural design of these systems on performance. The experiments also highlight the overhead of all-or-nothing atomicity, concurrency control, constraint enforcement, replication semantics, consistent queries, audit logging, and asynchronous processing of events. **It is worth**

**noting these cannot be obtained by existing microservice benchmarks (§ 2)**. Furthermore, from the lessons learned obtained through *Online Marketplace*, we derive design decisions to advance the development of data systems, highlighting that *Online Marketplace* will support designing futuristic data management systems for microservices.

## Acknowledgments

This work was partially supported by Independent Research Fund Denmark under Grant 9041-00368B. All of the computations for this project were performed on the UCloud interactive HPC system.
Dedicated to the memory of José Apolinário Nunes †.

## References

[1] Materialize [n.d.]. *Materialize: The Operational Data Warehouse.* Materialize. Retrieved December 12, 2023 from https://materialize.com
[2] 2018. *InconsistentStateException with ADO.NET SQL Server grain storage.* Retrieved November 25, 2023 from https://github.com/dotnet/orleans/issues/4697#issuecomment-398556401
[3] 2023. *Cluster management in Orleans.* Retrieved November 25, 2023 from https://learn.microsoft.com/en-us/dotnet/orleans/implementation/cluster-management
[4] 2023. *Grain persistence.* Retrieved November 25, 2023 from https://learn.microsoft.com/en-us/dotnet/orleans/grains/grain-persistence/?pivots=orleans-7-0
[5] Olist 2024. *Brazilian E-Commerce Public Dataset by Olist.* Olist. Retrieved September 5, 2024 from https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
[6] DuckDB 2024. *DuckDB.* DuckDB. Retrieved April 12, 2024 from https://github.com/duckdb
[7] Microsoft 2024. *Microservice Applications.* Microsoft. Retrieved September 16, 2024 from https://azure.microsoft.com/en-us/solutions/microservice-applications
[8] Amazon AWS 2024. *Microservices.* Amazon AWS. Retrieved September 16, 2024 from https://aws.amazon.com/microservices
[9] Alibaba Cloud 2024. *Microservices Engine.* Alibaba Cloud. Retrieved September 16, 2024 from https://www.alibabacloud.com/en/product/microservices-engine
[10] Google Cloud 2024. *What is Microservices Architecture?* Google Cloud. Retrieved September 16, 2024 from https://cloud.google.com/learn/what-is-microservices-architecture
[11] Andreas Andreakis, Falguni Jhaveri, Ioannis Papapanagiotou, Mark Cho, Poorna Reddy, and Tongliang Liu. 2019. *Delta: A Data Synchronization and Enrichment Platform.* Netflix. Retrieved April 8, 2024 from https://netflixtechblog.com/delta-a-data-synchronization-and-enrichment-platform-e82c36a79aee
[12] Anonymous. 2024. *A Benchmark for Data Management Challenges in Microservices.* https://zenodo.org/records/12731873 Version with Appendix.
[13] Arvind Arasu, Mitch Cherniack, Eduardo Galvez, David Maier, Anurag S. Maskey, Esther Ryvkina, Michael Stonebraker, and Richard Tibbetts. 2004. Linear Road: A Stream Data Management Benchmark. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30* (Toronto, Canada) *(VLDB '04)*. VLDB Endowment, 480–491.
[14] BroadleafCommerce. [n.d.]. *Broadleaf Microservices Architecture.* Broadleaf Commerce. Retrieved March 24, 2024 from https://www.broadleafcommerce.com/product/microservice-architecture
[15] Sebastian Burckhardt, Badrish Chandramouli, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, Christopher S. Meiklejohn, and Xiangfeng Zhu. 2022. Netherite: efficient execution of serverless workflows. *Proc. VLDB Endow.* 15, 8 (April 2022), 1591–1604. https://doi.org/10.14778/3529337.3529344
[16] Sergey Bykov, Alan Geller, Gabriel Kliot, Jim Larus, Ravi Pandya, and Jorgen Thelin. 2010. *Orleans: A Framework for Cloud Computing.* Technical Report MSR-TR-2010-159. https://www.microsoft.com/en-us/research/publication/orleans-a-framework-for-cloud-computing/
[17] Marcus Cavalcanti. 2020. *Lessons learned about running Microservices.* B2W. Retrieved April 8, 2024 from https://medium.com/b2w-engineering-en/lessons-learned-about-running-microservices-a51d952cb50b
[18] Brian Chavez. 2024. *Bogus.* Microsoft. Retrieved April 8, 2024 from https://github.com/bchavez/Bogus Bogus is fundamentally a C# port of faker.js and inspired by FluentValidation's syntax sugar.
[19] Jon Chew. 2019. *Avoiding Double Payments in a Distributed Payments System.* AirBnb. Retrieved April 8, 2024 from https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb

[20] Dmitry Chornyi. 2018. *Engineering Uber's Next-Gen Payments Platform.* Uber Technologies Inc. Retrieved April 8, 2024 from https://eng.uber.com/payments-platform

[21] Richard Cole, Florian Funke, Leo Giakoumakis, Wey Guy, Alfons Kemper, Stefan Krompass, Harumi Kuno, Raghunath Nambiar, Thomas Neumann, Meikel Poess, Kai-Uwe Sattler, Michael Seibold, Eric Simon, and Florian Waas. 2011. The mixed workload CH-benCHmark. In *Proceedings of the Fourth International Workshop on Testing Database Systems* (Athens, Greece) *(DBTest '11)*. Association for Computing Machinery, New York, NY, USA, Article 8, 6 pages. https://doi.org/10.1145/1988842.1988850

[22] Dan Conger. 2021. *How Apache Kafka Enables Podium to "Ship It and See What Happens"*. Confluent. Retrieved April 8, 2024 from https://www.confluent.io/blog/how-apache-kafka-enables-podium-to-ship-it-and-see-what-happens

[23] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) *(SoCC '10)*. Association for Computing Machinery, New York, NY, USA, 143–154. https://doi.org/10.1145/1807128.1807152

[24] TPC Council. 2005. *TPC-W: Benchmarking An Ecommerce Solution.* https://www.tpc.org/tpcw/tpc-w_wh.pdf Revision 1.2.

[25] TPC Council. 2010. *TPC Benchmark C revision.* https://www.tpc.org/tpcc Revision 5.11.

[26] Jérôme Darmont and Michel Schneider. 2002. Object-Oriented Database Benchmarks. In *Advanced Topics in Database Research, Vol. 1*, Keng Siau (Ed.). Idea Group, 34–57.

[27] Datadog. [n.d.]. *What Is Audit Logging?* Retrieved November 20, 2023 from https://www.datadoghq.com/knowledge-center/audit-logging/#audit-logs-vs-regular-system-logs

[28] Stephan Ewen. 2020. *Stateful Functions 2.0 - An Event-driven Database on Apache Flink.* Retrieved November 25, 2023 from https://flink.apache.org/2020/04/07/stateful-functions-2.0-an-event-driven-database-on-apache-flink

[29] Rafael Ferreira. 2019. *Microservices at Nubank, an overview.* Nubank. Retrieved April 8, 2024 from https://building.nubank.com.br/microservices-at-nubank-an-overview

[30] Rafael Ferreira and Edward Wible. 2017. *Architecting a Modern Financial Institution.* Nubank. Retrieved April 8, 2024 from https://www.infoq.com/presentations/nubank-architecture

[31] Apache Flink. 2024. *Flink Architecture.* Apache Flink. Retrieved September 23, 2024 from https://nightlies.apache.org/flink/flink-docs-master/docs/concepts/flink-architecture

[32] Martin Fowler. 2017. *What do you mean by "Event-Driven"?* Retrieved November, 1st 2023 from https://martinfowler.com/articles/201701-event-driven.html

[33] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 3–18.

[34] Seth Gilbert and Nancy Lynch. 2002. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *SIGACT News* 33, 2 (jun 2002), 51–59. https://doi.org/10.1145/564585.564601

[35] Adam Gluck. 2022. *Introducing Domain-Oriented Microservice Architecture.* Uber Technologies Inc. Retrieved April 8, 2024 from https://www.uber.com/en-DK/blog/microservice-architecture

[36] IATA. [n.d.]. *Airline Retailing.* IATA. Retrieved March 24, 2024 from https://www.iata.org/en/programs/airline-distribution/retailing

[37] imbursepayments. [n.d.]. *What is a PSP and how does it work?* Retrieved November, 4 2022 from https://imbursepayments.com/what-is-a-psp

[38] Ninad Khisti. 2018. *Measuring Transactional Integrity in Airbnb's Distributed Payment Ecosystem.* AirBnb. Retrieved April 8, 2024 from https://medium.com/airbnb-engineering/measuring-transactional-integrity-in-airbnbs-distributed-payment-ecosystem-a670d6926d22

[39] Martin Kleppmann, Alastair R. Beresford, and Boerge Svingen. 2019. Online Event Processing. *Commun. ACM* 62, 5 (apr 2019), 43–49. https://doi.org/10.1145/3312527

[40] Rodrigo Laigner. 2023. *EventBenchmark.* Retrieved April 4, 2024 from https://github.com/diku-dk/EventBenchmark/releases/tag/v1.0

[41] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. 2021. Data Management in Microservices: State of the Practice, Challenges, and Research Directions. *Proc. VLDB Endow.* 14, 13 (sep 2021), 3348–3361. https://doi.org/10.14778/3484224.3484232

[42] Yijian Liu, Rodrigo Laigner, and Yongluan Zhou. 2024. Rethinking State Management in Actor Systems for Cloud-Native Applications. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) *(SoCC '24)*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3698038.3698540

[43] Yijian Liu, Li Su, Vivek Shah, Yongluan Zhou, and Marcos Antonio Vaz Salles. 2022. Hybrid Deterministic and Nondeterministic Execution of Transactions in Actor Systems. In *Proceedings of the 2022 International Conference on*

*Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 65–78. https://doi.org/10.1145/3514221.3526172

[44] Materialize. [n.d.]. *How Ecommerce Company Drizly uses Materialize for Real-Time Notifications, Alerting, and Personalization.* Retrieved November, 4 2022 from https://materialize.com/customer-stories/drizly

[45] Math.NET. 2023. *Math.NET Numerics.* Retrieved November 20, 2023 from https://numerics.mathdotnet.com/

[46] Ricardo Mayerhofer. 2018. *restQL: Tackling microservice query complexity.* B2W. Retrieved April 8, 2024 from https://medium.com/b2w-engineering-en/restql-tackling-microservice-query-complexity-27def5d09b40

[47] Osvaldo Santana Neto. 2018. *Olist Architecture From Monolith to microservices.* Olist. Retrieved March 18, 2024 from https://www.slideshare.net/osantana/olist-architecture-v20

[48] Matheus Oliveira. 2017. *Gerenciamento de dados com micro-serviços e a experiência do iFood.* iFood. Retrieved April 8, 2024 from https://www.infoq.com/br/presentations/gerenciamento-de-dados-com-micro-servicos-e-a-experiencia-do-ifood

[49] Microsoft Orleans. 2023. *Request scheduling.* Retrieved November 25, 2023 from https://learn.microsoft.com/en-us/dotnet/orleans/grains/request-scheduling

[50] Microsoft Orleans. 2024. *Grain placement.* Microsoft. Retrieved September 23, 2024 from https://learn.microsoft.com/en-us/dotnet/orleans/grains/grain-placement#random-placement

[51] Microsoft Orleans. 2024. *Orleans streams implementation details.* Microsoft. Retrieved April 8, 2024 from https://learn.microsoft.com/en-us/dotnet/orleans/implementation/streams-implementation/

[52] Microsoft Orleans. 2024. *Streaming with Orleans.* Microsoft. Retrieved March 24, 2024 from https://learn.microsoft.com/en-us/dotnet/orleans/streaming/?pivots=orleans-7-0

[53] Dan Pritchett. 2008. Base: An Acid Alternative. In *File Systems and Storage*, Vol. 6. ACM Queue. Issue 3.

[54] Nitin Sarma. 2020. *Event Driven Architectures @ Netflix Content Finance Engineering.* Netflix. Retrieved April 8, 2024 from https://www.linkedin.com/pulse/event-driven-architectures-netflix-content-finance-engineering-s/

[55] Vivek Shah and Marcos Antonio Vaz Salles. 2018. Reactors: A Case for Predictable, Virtualized Actor Database Systems. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 259–274. https://doi.org/10.1145/3183713.3183752

[56] Natan Silnitsky. 2022. *Event Driven Architecture — 5 Pitfalls to Avoid.* Wix. Retrieved July 1, 2023 from https://medium.com/wix-engineering/event-driven-architecture-5-pitfalls-to-avoid-b3ebf885bdb1

[57] Natan Silnitsky. 2022. *Troubleshooting Kafka for 2000 Microservices at Wix.* Wix. Retrieved July 1, 2023 from https://medium.com/wix-engineering/troubleshooting-kafka-for-2000-microservices-at-wix-986ee382fd1e

[58] Aakriti Singla and Simon Wu. 2020. *Revolutionizing Money Movements at Scale with Strong Data Consistency.* Uber Technologies Inc. Retrieved April 8, 2024 from https://eng.uber.com/money-scale-strong-data

[59] Apache Flink Statefun. 2023. *Co-located Functions.* Retrieved November 25, 2023 from https://nightlies.apache.org/flink/flink-statefun-docs-master/docs/concepts/distributed_architecture/#co-located-functions

[60] Apache Flink Statefun. 2023. *Distributed Architecture.* Retrieved November 25, 2023 from https://nightlies.apache.org/flink/flink-statefun-docs-master/docs/concepts/distributed_architecture

[61] Apache Flink Statefun. 2023. *Shopping Cart Example with Docker Compose.* Retrieved November 25, 2023 from https://github.com/apache/flink-statefun-playground/tree/main/java/shopping-cart

[62] Apache Flink Statefun. 2023. *Stateful Functions: A Platform-Independent Stateful Serverless Stack.* Retrieved November 11, 2023 from https://nightlies.apache.org/flink/flink-statefun-docs-master/

[63] Statista. 2024. *eCommerce - Worldwide.* Retrieved March 18, 2024 from https://www.statista.com/outlook/emo/ecommerce/worldwide

[64] Jan Stenberg. 2019. *Experiences Moving from Microservices to Workflows at Jet.com.* Jet.com. Retrieved March 21, 2024 from https://www.infoq.com/news/2019/02/migrate-microservices-workflows

[65] Rebecca Taft, Nosayba El-Sayed, Marco Serafini, Yu Lu, Ashraf Aboulnaga, Michael Stonebraker, Ricardo Mayerhofer, and Francisco Andrade. 2018. P-Store: An Elastic Database System with Predictive Provisioning. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 205–219. https://doi.org/10.1145/3183713.3190650

[66] UCloud. [n.d.]. *Products - UCloud.* Retrieved November 28, 2023 from https://docs.cloud.sdu.dk/guide/resources-products.html#compute

[67] UCloud. [n.d.]. *UCloud User Guide.* Retrieved November 28, 2023 from https://docs.cloud.sdu.dk

[68] Nicolas Viennot, Mathias Lécuyer, Jonathan Bell, Roxana Geambasu, and Jason Nieh. 2015. Synapse: A Microservices Architecture for Heterogeneous-Database Web Applications. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) *(EuroSys '15)*. Association for Computing Machinery, New York, NY, USA, Article 21, 16 pages. https://doi.org/10.1145/2741948.2741975

[69] VTEX. [n.d.]. *Commerce Microservices.* VTEX. Retrieved March 18, 2024 from https://vtex.com/us-en/commerce-microservices/

[70] Yiwen Wang, Júlio Cesar dos Reis, Kasper Myrtue Borggren, Marcos Antonio Vaz Salles, Claudia Bauzer Medeiros, and Yongluan Zhou. 2019. Modeling and Building IoT Data Platforms with Actor-Oriented Databases. In *Advances in Database Technology - 22nd International Conference on Extending Database Technology, EDBT 2019, Lisbon, Portugal, March 26-29, 2019*. OpenProceedings.org, 512–523. https://doi.org/10.5441/002/edbt.2019.47

[71] Yang Yang, Zhifeng Chen, Qichao Chu, Haitao Zhang, and George Teo. 2021. *Enabling Seamless Kafka Async Queuing with Consumer Proxy*. Uber Technologies Inc. Retrieved April 8, 2024 from https://www.uber.com/en-SE/blog/kafka-async-queuing-with-consumer-proxy/

[72] Chao Zhang, Guoliang Li, and Tao Lv. 2024. HyBench: A New Benchmark for HTAP Databases. *Proc. VLDB Endow.* 17, 5 (May 2024), 939–951. https://doi.org/10.14778/3641204.3641206

[73] Chao Zhang, Jiaheng Lu, Pengfei Xu, and Yuxing Chen. 2019. UniBench: A Benchmark for Multi-model Database Management Systems: Recognizing Outstanding Ph.D. Research. 7–23. https://doi.org/10.1007/978-3-030-11404-6_2

[74] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Softw. Eng.* 47, 2 (feb 2021), 243–260. https://doi.org/10.1109/TSE.2018.2887384

[75] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 323–324. https://doi.org/10.1145/3183440.3194991

Received July 2024; revised September 2024; accepted November 2024